T-504-ITML: Machine Learning
Supervised Learning for Insurance Fraud Detection

Kristófer O. Svavarsson

Reykjavík University

23 March 2026

1 Introduction

Fraud in the car insurance industry results in significant problems each year, making accurate and efficient detection methods a priority. Traditional approaches are often resource-heavy because they are based on manual investigation. This paper explores the use of machine learning as an easier and scalable solution for detecting fraudulent car insurance claims. Machine learning models, capable of automatically identifying patterns and anomalies, can learn from historical data and generalize these insights to new claims. In this experiment, six machine learning models are evaluated with performance metrics such as accuracy, precision, and recall, to determine their effectiveness in distinguishing fraudulent activity from legitimate claims.

2 Data description

The car insurance fraud dataset used in this experiment was retrieved from Keggle, an online platform for data scientists and machine learning practitioners under Google LLC that holds extensive amounts of data on various topics. The dataset has a total of 32 independent variables and one target variable, that specifies whether or not a fraud was found.

3 Process

3.1 Preprocessing

3.1.1 Irrelevant and redundant attributes

Usually, not all attributes in a dataset apply to predicting the target variable, and the only appropriate option is to remove them. Here, 12 variables were removed from the insurance fraud dataset due to irrelevance. These include variables related to time series analysis, such as the year, month, day of the week, etc., and IDs, i.e. 'Policy number' and 'Rep number'. 2 attributes were removed due to redundancy: 'Age' (a duplicate of 'Age Of Policy Holder'), and 'Policy Type' (a combination of 'Vehicle Category' and 'Base Policy'). A list of variables removed from the dataset can be found in Table 1 below.

3.1.2 Missing values

The insurance fraud dataset contained some features with missing values. These were 'Number Of Supplements', 'Past Number Of Claims', 'Days Policy Claim', and 'Days Policy Accident'. The removal of observations with missing values is an approach that is sometimes taken to address this. However, this is only viable if the percentage of observations that have values missing is rather small. Since some of the features in the insurance fraud dataset contain a high

Category	Attributes
IDs	1. Policy Number 2. Rep Number
Variables related to time series analysis	3. Month 4. Week Of Month 5. Day Of Week 6. Day Of Week Claimed 7. Month Claimed 8. Week Of Month Claimed 9. Year
Duplicates	10. Age 12. Policy Type
Obviously irrelevant attributes	13. Address Change Claim 14. Make 15. Deductible

Table 1: Removed Variables

number of missing values (up to about 7000) it was decided to fill them in automatically instead. For each missing value, the most common value in that category was used.

3.1.3 Feature selection

The dimensionality of the dataset is often a concern of researchers in the machine learning field. If the dimensionality is large, it can be difficult to detect dependencies between the target variable and the independent variables. Hence, it is important to consider only those features that have a high chance of being correlated with the target variable. Faheem et al. performed feature selection with an R package known as Boruta for detecting the most significant features on a nearly identical car insurance fraud dataset from the decision analytics department of an American firm (EXL Service). They found nine features to be significant for predicting insurance fraud: 'sex', 'days policy claim', 'policy type', 'marital status', 'vehicle category', 'age of vehicle', 'age of policyholder', 'base policy', and 'fault', with 'fault' being by far the most significant feature. (Aslam et al.) Hence, only those features, except for 'policy type' (since it is the combination of base policy and vehicle category, were used in this experiment, a total of 7.

3.1.4 Data type conversion

Many machine learning models in Scikit-learn, like the ones selected for this experiment, cannot handle string values directly. The models expect numerical data, and the insurance fraud dataset has features that contain strings as categorical values. To address this, the data was encoded into integers. For DT and SVM, Scikit-learn's *LabelEncoder* was used. For other models, Pandas' *getdummies* function for One-Hot Encoding was used.

3.2 Model selection

A total of six different machine-learning models were chosen for this experiment. The models were chosen based on which ones were covered in the course, how likely they were to be good, and their prevalence in the literature. Faheem et al, for example, used both Naïve Bayes and SVM for auto insurance fraud detection, and Ain et al. (2020) found K-nearest neighbors to have stronger performance detecting credit card frauds compared to Naïve Bayes, LR, RF, DT, SVM, J48 and Binary Classification Technique (BCT). GBDT is likewise an excellent model both for regression and classification, in particular for tabular data (Scikit-learn).

3.2.1 K-nearest neighbors - KNN

K-Nearest Neighbors (KNN) is an instance-based classification algorithm, meaning it doesn't rely on an explicit training phase like other models. Instead of building a model, it compares new instances directly to existing observations to make predictions.

One of KNN's main advantages is that it is easy to implement and requires only a few hyperparameters to tune. The key hyperparameters include the number of neighbors (k) and the choice of a distance metric, which measures

the similarity between instances. Furthermore, KNN is highly adaptable, making it suitable for a wide variety of applications.

Scikit-learn provides two different kinds of KNN implementations: *KNeighborsClassifier* is based on the K nearest neighbors of each query point, where K is an integer value specified by the user. *RadiusNeighborsClassifier* implements learning based on the number of neighbors within a fixed radius R of each training point, where R is a floating-point value specified by the user.

KNeighborsClassifier was the chosen kind owing to the fact that it is the more commonly used technique (Scikit-learn).

Sources: KNN lecture slides, Scikit-learn (https://scikit learn.org/stable/modules/neighbors.html)

3.2.2 Naïve bayes

Naïve Bayes (NB) is a probabilistic framework for solving classification problems. Bayes' theorem is used as a basis for prediction with the "naive" assumption that all features are conditionally independent given the class label. Bayes' theorem gives us the following relationship, given class variable Y and dependent feature vector X_1 through X_n:

$$P(Y \mid X_1, X_2, \ldots, X_n) = \frac{P(X_1, X_2, \ldots, X_n \mid Y)P(Y)}{P(X_1, X_2, \ldots, X_n)}$$

which is predicated on the naive conditional independence assumption that:

$$P(X_1, X_2, \ldots, X_n \mid Y_j) = P(X_1 \mid Y_j)P(X_2 \mid Y_j) \ldots P(X_n \mid Y_j)$$

In this experiment, there are eight dependent features shown in Table X with the independent variable being 'FraudFoundP'.

Scikit-learn provides several different NB classifiers, of which *CategoricalNB* is of particular note. It assumes that each feature, described by the index i, has its own categorical distribution. Because the insurance fraud data is categorically distributed, meaning each feature in the dataset is discrete, the *CategoricalNB* classifier was used in this experiment.

Sources: https://scikit-learn.org/stable/modules/naive$_b ayes.htmlNBlectureslides$

3.2.3 Decision tree

A decision tree (DT) is a (non-parametric supervised learning method) that can be used for classification. DT uses a divide-and-conquer approach, recursively partitioning the data into decision regions (nodes), where each leaf node represents a prediction. The recursive partitioning is repeated until all, or the majority, of records have been classified. Finding the optimal split depends mainly on the purity of the class (i.e. how segregated the split is), called the gini-coefficient.

Scikit-learn's (only) implementation of DT for classification is the *DecisionTreeClassifier*.

Sources: https://www.ibm.com/topics/decision-trees, https://scikit-learn.org/stable/modules/tree.html lecture slides

3.2.4 Gradient boosted tree

Gradient Boosted Decision Trees (GBDT) use a sequential process that takes data and a differentiable loss function as input. The goal is to iteratively make new models that improve the previous models' predictions by minimizing the loss function.

Scikit-learn provides two implementations of gradient-boosted trees for classification: *HistGradientBoostingClassifier* and *GradientBoostingClassifier*. *HistGradientBoostingClassifier* is a much faster variant of GradientBoostingClassifier for intermediate and large datasets, where the number of samples is at least 10.000. Hence it is the one used here.

(evaluating different models) HistGradientBoostingClassifier does not rely on traditional resampling methods like random sampling or stratified random sampling, which are common in other ensemble methods like bagging or standard gradient boosting. Instead, it uses randomized subsampling with a focus on efficiency and performance

Sources: https://scikit-learn.org/stable/modules/ensemble.html lecture slides

3.2.5 Random forest

The Random Forest (RF) classifier is a special type of ensemble learning method that combines multiple bagged decision trees. In this technique, each decision tree in the ensemble is trained not only on a randomly selected subset of the observations (as in traditional bagging) but also on a random subset of the features. This process is sometimes referred to as feature bagging. The selection of a subset of features is done with replacement for each individual tree, meaning that every tree is constructed using only a portion of the overall feature set.

During the training process, each tree attempts to find the best split among its chosen subset of features, creating a diverse set of trees within the forest. This diversity helps improve the overall model's performance, as different trees focus on different aspects of the data. By aggregating the predictions from all the trees, the Random Forest classifier typically achieves better accuracy and robustness than a single decision tree.

Scikit-learn's RF implementation is the *RandomForestClassifier*. Each tree in the ensemble is built from a sample drawn with replacement (i.e., a bootstrap sample) from the training set. Scikit-learn has another version of RF called *ExtraTreesClassifier* that goes one step further in the way splits are computed, where, instead of looking for the most discriminative thresholds, thresholds are drawn at random for each candidate feature and the best of these randomly-generated thresholds is picked as the splitting rule. This usually reduces the variance of the model, at the expense of a slightly greater increase in bias. *ExtraTreesClassifier* was not considered for this experiment on account of RandomForestClassifier having sufficiently low variance on the insurance fraud dataset.

Sources: Random forest lecture slides, https://scikit-learn.org/stable/modules/ensemble.htmlrandom-forests-and-other-randomized-tree-ensembles

3.2.6 Support vector machine - SVM

Support Vector Machine (SVM) is one of the best choices for binary classification problems (Aslam et. al.). SVM distinguishes between two classes ("FraudFaund" and "NoFraud" in this case) by finding the optimal hyperplane that maximizes the margin between the closest data points of opposite classes. (https://www.ibm.com/topics/support-vector-machine) Using the SV technique, one can solve problems that in classical techniques would require estimating a large number of free parameters (Golowich et al.).

SVC is an implementation of SVM in Scikit-learn capable of performing both binary and multi-class classification on a dataset, and it was the one used in this experiment. Scikit-learn provides two other implementations: NuSVC and LinearSVC. SVC and NuSVC are similar methods but accept slightly different sets of parameters and have different mathematical formulations. On the other hand, LinearSVC scales better to large numbers of samples. LinearSVC was not used, however, as a result of fitting failures of the model to the data.

3.3 Finding the best model

The process for finding the best model was two-fold. First, the best combination of hyperparameters for each model was found. This was done by repeatedly applying cross-validation with different combinations of hyperparameters, recording the combination with the best accuracy. Virtually every hyperparameter available for each of the models was considered. The number of values for the parameters ranged from two to four and were mostly chosen in an ordinal fashion, including what can be considered a small value, a medium-sized value, and a large value for each of the parameters, in addition to the default value that the parameter takes. The 'class weight' parameter was not tested with a custom class weight. To save time, only the default value of 'None' and 'balanced' were tested.

SVM was an exception to this, however. As a consequence of slow testing time, a handful of hyperparameters were not considered for testing. This includes 'kernel'.

SVC in Scikit-learn has different types of kernels available, like linear, poly, rbf, and sigmoid. Only the default kernel rfb was used.

An exhaustive search was performed on KNN and NB with Sciki-learn's GridSearchCV algorithm. In service of time, however, Sciki-learn's RandomizedSearchCV was used for all other models.

Next, the best models were evaluated using cross-validation once again, to get additional performance measures, like precision and recall. A list of the performance measures is provided in table 2.

Evaluation metric	**Formula**
Accuracy	$Acc = \frac{TP+TN}{TP+FP+TN+FN}$
Precision	$P = \frac{TP}{TP+FP}$
Recall	$R = \frac{TP}{TP+FN}$

Table 2: Performance Metrics Definitions

4 Results

Evaluation of the models is done by computing confusion matrices, a common way to show model performance, as well as comparing tree performance metrics: **Accuracy** is the Share of correctly classified instances (both true positives and true negatives) to the total instances in the dataset. It provides an overall measure of how well the model is performing. **Precision** measures the share of positive predictions that the model predicted correctly. And **Recall** shows how often the model gets the fraudulent cases right among just the fraudulent cases. I.e. it measures the ratio of correctly predicted positive observations (true positives) to all actual positive observations.

Model	**Test dataset confusion metrics**	**Predicted not Fraud**	**Predicted Fraud**
KNN	Actual not fraud	2882	0
	Actual Fraud	202	0
NB	Actual not fraud	2879	3
	Actual Fraud	199	3
DT	Actual not fraud	2882	0
	Actual Fraud	202	0
GBDT	Actual not fraud	2882	0
	Actual Fraud	202	0
RF	Actual not fraud	2882	0
	Actual Fraud	202	0
SVM	Actual not fraud	2882	0
	Actual Fraud	202	0

Table 3: Confusion Matrices

Model	**Accuracy**	**Precision**	**Recall**
KNN	0.9414720994600362	0	0
NB	0.9400941044149056	0.5	0.01485148514851485
DT	0.941553202434206	0	0
GBDT	0.941553202434206	0	0
RF	0.941553202434206	0	0
SVM	0.941553202434206	0	0

Table 4: Performance Metrics

Accuracy, Precision, and Recall can be found in Table 4. The results show that the models are all very similar in terms of accuracy. The highest accuracy among the six machine learning models is achieved by tree models, all sharing the same score of approximately 94.155%. NB shows the lowest accuracy of about 94.009%. Overall, the models have very high accuracy. The same can not be said about precision and recall, however. Every model except Naïve Bayes received a precision and recall score of zero.

The 2x2 confusion matrices of the six machine learning models is given in Table 3. The confusion matrix is a matrix whose i-th row and j-th column entry indicates the number of samples with the true label being i-th class and

the predicted label being j-th class, meaning that correct predictions fall diagonally straight through the middle of the matrix.

The findings show that the models are heavily biased towards not predicting fraud. No model, except Naïve Bayes, predicted fraud out of the 3084 instances they were tasked with predicting, classifying all 202 fraudulent cases as non-fraudulent. Naïve Bayes shows a slightly better performance, predicting 3 instances fraudulent and classifying them correctly.

5 Conclusions

Car insurance fraud detection is a difficult task, but an important one nonetheless. Overall, the results leave a lot to be desired. A precision and recall scores of zero in the case of all models except NB indicate that they are severely overfitting the data. Faheem et al. obtained somewhat better results using three predictive models: Linear Regression, Naïve Bayes, and SVM. The highest accuracy was achieved by SVM, 94.002%, and Naïve Bayes came in close second place with an accuracy of 93.303%. In terms of precision, SVM had the lowest score of 16.667% and Naïve Bayes the highest, 23.077%. Concerning recall, neither SVM nor Naïve Bayes had optimal scores, 0.704% and 6.338%, respectively.

In this experiment, KNN, DT, GBDT, RF, and SVM all got the sub-optimal score of zero in terms of precision and recall. This means that these five models failed to correctly classify any instances containing fraud, leading to an overall lack of effectiveness in classifying the target class. In contrast, the Naïve Bayes model, while achieving lower accuracy, managed to produce non-zero precision and recall scores. This suggests that, although its overall accuracy is slightly lower, it may be more adept than the other models at identifying relevant instances when they occur.

The disparity between accuracy and precision/recall metrics highlights a critical aspect of model evaluation. While accuracy provides a broad overview of performance, it can be misleading in scenarios with imbalanced classes or when the cost of false negatives is high. In this case, the high accuracy of the six models does not correlate with their ability to identify positive instances effectively.

6 Future work

The low precision and recall scores of the five models indicate issues related to overfitting. It may be necessary to revisit the feature selection process, as the identification and removal of anomalies in the dataset was not done. There might be anomalies in the data acting as noise that could have distorted the outcome of the tests.

Furthermore, it would likely be beneficial to explore more pre-processing techniques such as feature scaling, in which data is transformed to fit within a more specified range to ensure that all features are on the same scale.

Finally, the dataset itself is quite old. The observations were collected from 1994 to 1996, and it may be appropriate to look at newer data that may be more applicable for predicting car insurance fraud today.

References

[1] Aslam, F., Hunjra, A. I., Ftiti, Z., Shams, T. (2022) *Insurance fraud detection: Evidence from artificial intelligence and machine learning*. Research in International Business and Finance. Volume 62.

[2] Ain, Q. U., Zia, M. A., Asghar, N., Saleem, A. (2022) *Analysis of Variant Data Mining Methods for Depiction of Fraud*. Research in International Business and Finance. Springer, p. 423–432.

[3] Golowich, S. E., Smola, A., Vapnik, V. (1995) *Support Vector Method for Function Approximation, Regression Estimation, and Signal Processing*. NIPS Papers.